CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

University Institute of Computing

PYTHON PROGRAMMING LAB

PROJECT REPORT

SALES DATA ANALYSIS SYSTEM

Submitted By	Craig Mapungwana
UID	24BCA 10325
Class / Section	3B
Subject	Python Programming Lab
Project Title	Sales Data Analysis System
Academic Session	2026·2027

Submitted in partial fulfilment of the requirements of the Python Programming Lab.

CERTIFICATE

This is to certify that the project report entitled **“Sales Data Analysis System”** has been prepared as part of the Python Programming Lab coursework. The project demonstrates the use of Python programming constructs to read, process, analyse and report retail sales information.

STUDENT DECLARATION

I declare that this report describes the design and implementation of a Python-based sales analysis application. The program, dataset structure, calculations and outputs are documented in the following pages.

ACKNOWLEDGEMENT

I would like to express my gratitude to the faculty members of Chandigarh University for their guidance and for providing the opportunity to apply Python programming concepts through a practical project.

ABSTRACT

The Sales Data Analysis System is a menu-driven Python application developed to help a retail business understand its sales performance. The system imports records from a comma-separated values (CSV) file, validates the input, calculates revenue, aggregates quantities and revenue by product and category, reports daily revenue, supports product searches and exports a summary report. The project applies functions, loops, conditional statements, dictionaries, lists, file handling, exception handling and standard-library modules. A small sample dataset is included to demonstrate the complete workflow.

INDEX / TABLE OF CONTENTS

S. No.	Section	Page
1	Introduction and Background	3
2	Problem Statement	3
3	Objectives and Scope	4
4	Hardware and Software Requirements	4
5	System Analysis and Design	5
6	Algorithm and Workflow	6
7	Dataset Description	7
8	Python Concepts and Modules	8
9	Implementation and Source Code	9
10	Program Execution and Screenshots	12
11	Testing and Results	16
12	Limitations and Future Scope	17

S. No.	Section	Page
13	Conclusion	18
14	References	18

1. INTRODUCTION AND BACKGROUND

The growth of digital retail operations has increased the amount of information generated through daily transactions. Sales records contain useful details such as transaction date, order identifier, product, category, quantity and unit price. When these records are examined systematically, a business can understand which products are performing well, how much revenue is being generated and how sales vary over time.

This project applies Python to transform raw transaction rows into useful summaries. Rather than manually calculating every sale, the application reads the dataset and performs repeatable calculations automatically. The program is designed as a small educational prototype that demonstrates the workflow of a basic data-analysis system.

2. PROBLEM STATEMENT

A retail store may maintain sales records in a spreadsheet or CSV file but still lack a quick way to calculate total revenue, compare products, identify the best-selling item and review category-wise or day-wise performance. Manual calculations are time-consuming and can lead to inconsistent results.

The proposed system addresses this problem by loading sales data from a structured CSV file, validating each row, calculating revenue for each record and presenting summaries through a simple text-based menu.

3. OBJECTIVES AND SCOPE

Objectives: (1) read sales data from a CSV file; (2) validate dates, quantities and prices; (3) calculate revenue as quantity multiplied by unit price; (4) calculate total units and total revenue; (5) compare products by quantity and revenue; (6) analyse categories and daily revenue; (7) search for a product; and (8) save an analysis report.

Scope: The current version is a command-line prototype using a local CSV dataset. It is suitable for learning and demonstration. It does not connect to a live point-of-sale system, database or online store.

4. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware: a standard desktop or laptop computer, keyboard, display and sufficient storage for Python and project files. No specialised hardware is required.

Software: Windows, macOS or Linux; Python 3.x; and a code editor such as Visual Studio Code, IDLE or PyCharm. The application uses Python standard-library modules only, so no third-party package installation is required.

Project files: main.py contains the application; sales_data.csv contains input records; sales_analysis_report.txt is created when the report-generation menu option is selected.

5. SYSTEM ANALYSIS AND DESIGN

The application follows a simple input-process-output design. The CSV file is the input source. The loading function reads rows and validates fields. Processing functions calculate totals and group records. The menu acts as the user interface, and results are displayed in the terminal or written to a text report.

External entity: Store operator or student who supplies the CSV file and selects menu operations. **Process:** Sales Data Analysis System. **Data store:** sales_data.csv. **Output:** sales summaries, product/category comparisons, daily revenue and generated report.

6. ALGORITHM AND WORKFLOW

Algorithm:

1. Start the application.
2. Open the CSV file and read its column headings.
3. Check that all required columns exist.
4. For each row, convert the date, quantity and unit price into suitable data types.
5. Skip invalid rows and calculate revenue for valid rows.
6. Display the menu.
7. Execute the selected analysis function.
8. Repeat until the user chooses Exit.
9. End the program.

Revenue formula: Revenue = Quantity × Unit Price.
Total revenue: Sum of the revenue of all valid sales records.
Average order value: Total revenue ÷ number of unique order IDs (when at least one order exists).

7. DATASET DESCRIPTION

The sample CSV contains 12 sales records dated 1–6 October 2026. Each record has six fields: Date, Order_ID, Product, Category, Quantity and Unit_Price. The sample includes products from Electronics, Furniture and Stationery categories. The values are illustrative sample data used to demonstrate program functionality.

Keeping the headings unchanged is important because the program uses these names to access the corresponding fields. Dates are expected in YYYY-MM-DD format; quantity is an integer; and unit price is a numeric value.

8. PYTHON CONCEPTS AND MODULES

Variables and data types: strings identify products and categories, integers represent quantities, floats represent prices and revenue, and date objects represent transaction dates.

Functions: the program is divided into reusable functions such as load_sales(), sales_summary(), product_analysis() and save_report(). This makes the code easier to understand, test and maintain.

Conditional statements and loops: if/elif/else selects the requested menu operation; while repeats the menu; for iterates through records.

Collections: lists store sales records, dictionaries represent individual records, and defaultdict groups totals by product, category or date.

File handling: open() reads the CSV file and writes the generated text report. The with statement ensures files are closed correctly.

Exception handling: try/except prevents malformed rows or file-access problems from crashing the entire program.

Modules: csv handles comma-separated data, os checks file existence, collections provides defaultdict, and datetime parses dates.

9. IMPLEMENTATION AND SOURCE CODE

The implementation is organised into separate functions. load_sales() performs input and validation; display_records() prints transaction rows; sales_summary() calculates overall metrics; product_analysis() and category_analysis() group results; daily_revenue() aggregates sales by date; search_product() filters records; and save_report() exports a summary.

Menu selection is handled in main(). The application starts only when the file is executed directly, using the standard Python entry-point condition.

9.1 COMPLETE PYTHON SOURCE CODE

```
"""
SALES DATA ANALYSIS SYSTEM
Python Programming Lab Project
"""

import csv
import os
from collections import defaultdict
from datetime import datetime

DATA_FILE = "sales_data.csv"
REPORT_FILE = "sales_analysis_report.txt"


def load_sales(filename=DATA_FILE):
    """Read, validate and prepare sales records from a CSV file."""
    if not os.path.exists(filename):
        print(f"Error: {filename} was not found.")
        print("Keep sales_data.csv in the same folder as main.py.")
        return []

    records = []
    required = {"Date", "Order_ID", "Product", "Category", "Quantity", "Unit_Price"}
    try:
        with open(filename, "r", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            if not required.issubset(set(reader.fieldnames or [])):
                print("Error: required CSV columns are missing.")
                return []

            for row in reader:
                try:
                    sale_date = datetime.strptime(row["Date"], "%Y-%m-%d").date()
                    quantity = int(row["Quantity"])
                    unit_price = float(row["Unit_Price"])
                    if quantity < 0 or unit_price < 0:
                        raise ValueError("Negative quantity or price")
                    records.append({
                        "date": sale_date,
                        "order_id": row["Order_ID"],
                        "product": row["Product"],
                        "category": row["Category"],
                        "quantity": quantity,
                        "unit_price": unit_price,
                        "revenue": quantity * unit_price
                    })
                except (ValueError, TypeError):
                    print(f"Skipping invalid row: {row}")
    except OSError as error:
        print(f"Unable to read data: {error}")
    return records

def money(value):
    return f"Rs. {value:,.2f}"


def display_records(records):
    print("\\nSALES DATA RECORDS")
    print("-" * 88)
    print(f"{'Date':<13}{'Order ID':<11}{'Product':<19}{'Category':<15}{'Qty':>5}{'Price':>10}{'Revenue':>13}")
    print("-" * 88)
    for sale in records:
        print(f"{sale['date']!s:<13}{sale['order_id']:<11}{sale['product']:<19}"
              f"{sale['category']:<15}{sale['quantity']:>5}{sale['unit_price']:>10.2f}"
              f"{sale['revenue']:>13.2f}")
    print("-" * 88)
    print(f"Total records: {len(records)}")


def sales_summary(records):
    revenue = sum(s["revenue"] for s in records)
    quantity = sum(s["quantity"] for s in records)
    orders = len({s["order_id"] for s in records})
    average = revenue / orders if orders else 0
    print("\\nOVERALL SALES SUMMARY")
    print(f"Sales records       : {len(records)}")
    print(f"Unique orders       : {orders}")
    print(f"Units sold          : {quantity}")
    print(f"Total revenue       : {money(revenue)}")
    print(f"Average order value : {money(average)}")


def product_analysis(records):
```

```
    totals = defaultdict(lambda: {"quantity": 0, "revenue": 0.0})
    for sale in records:
        totals[sale["product"]]["quantity"] += sale["quantity"]
        totals[sale["product"]]["revenue"] += sale["revenue"]
    print("\\nPRODUCT PERFORMANCE")
    print(f"{'Product':<22}{'Quantity Sold':>15}{'Revenue':>18}")
    print("-" * 55)
    for product, values in sorted(totals.items(), key=lambda item: item[1]["revenue"], reverse=True):
        print(f"{product:<22}{values['quantity']:>15}{money(values['revenue']):>18}")
    if totals:
        best_quantity = max(totals, key=lambda name: totals[name]["quantity"])
        best_revenue = max(totals, key=lambda name: totals[name]["revenue"])
        print(f"Best-selling product (quantity): {best_quantity}")
        print(f"Highest revenue product       : {best_revenue}")


def category_analysis(records):
    totals = defaultdict(float)
    quantities = defaultdict(int)
    for sale in records:
        totals[sale["category"]] += sale["revenue"]
        quantities[sale["category"]] += sale["quantity"]
    print("\\nCATEGORY ANALYSIS")
    print(f"{'Category':<20}{'Units Sold':>14}{'Revenue':>18}")
    print("-" * 52)
    for category in sorted(totals, key=totals.get, reverse=True):
        print(f"{category:<20}{quantities[category]:>14}{money(totals[category]):>18}")


def daily_revenue(records):
    totals = defaultdict(float)
    for sale in records:
        totals[sale["date"]] += sale["revenue"]
    print("\\nDAILY REVENUE")
    print(f"{'Date':<18}{'Revenue':>18}")
    print("-" * 36)
    for day in sorted(totals):
        print(f"{day!s:<18}{money(totals[day]):>18}")


def search_product(records):
    term = input("Enter product name to search: ").strip().lower()
    matches = [s for s in records if term in s["product"].lower()]
    if not matches:
        print("No matching product records found.")
    else:
        display_records(matches)
        print("Matching revenue:", money(sum(s["revenue"] for s in matches)))


def save_report(records):
    revenue = sum(s["revenue"] for s in records)
    quantity = sum(s["quantity"] for s in records)
    products = defaultdict(lambda: {"quantity": 0, "revenue": 0.0})
    for sale in records:
        products[sale["product"]]["quantity"] += sale["quantity"]
        products[sale["product"]]["revenue"] += sale["revenue"]
    with open(REPORT_FILE, "w", encoding="utf-8") as file:
        file.write("SALES DATA ANALYSIS REPORT\\n")
        file.write("=" * 40 + "\\n")
        file.write(f"Sales records: {len(records)}\\n")
        file.write(f"Units sold: {quantity}\\n")
        file.write(f"Total revenue: {money(revenue)}\\n\\n")
        file.write("PRODUCT PERFORMANCE\\n")
        for name in sorted(products, key=lambda n: products[n]["revenue"], reverse=True):
            item = products[name]
            file.write(f"{name}: {item['quantity']} units | {money(item['revenue'])}\\n")
    print(f"Report successfully saved as '{REPORT_FILE}'.")


def main():
    records = load_sales()
    if not records:
        print("No valid sales records loaded. Check the CSV file.")
        return
    while True:
        print("\\n" + "=" * 48)
        print("            SALES DATA ANALYSIS SYSTEM")
        print("=" * 48)
        print("1. View all sales records")
        print("2. View overall sales summary")
        print("3. Analyse product performance")
        print("4. Analyse sales by category")
        print("5. View daily revenue")
```

```
        print("6. Search for a product")
        print("7. Generate and save analysis report")
        print("8. Exit")
        print("=" * 48)
        choice = input("Enter your choice (1-8): ").strip()
        if choice == "1":
            display_records(records)
        elif choice == "2":
            sales_summary(records)
        elif choice == "3":
            product_analysis(records)
        elif choice == "4":
            category_analysis(records)
        elif choice == "5":
            daily_revenue(records)
        elif choice == "6":
            search_product(records)
        elif choice == "7":
            save_report(records)
        elif choice == "8":
            print("Thank you for using the Sales Data Analysis System.")
            break
        else:
            print("Invalid choice. Please enter a number from 1 to 8.")


if __name__ == "__main__":
    main()
```

10. PROGRAM EXECUTION AND SCREENSHOTS

The following captures show the application output for the principal menu functions. Each screen corresponds to a run of the Python program using the included sample CSV file.

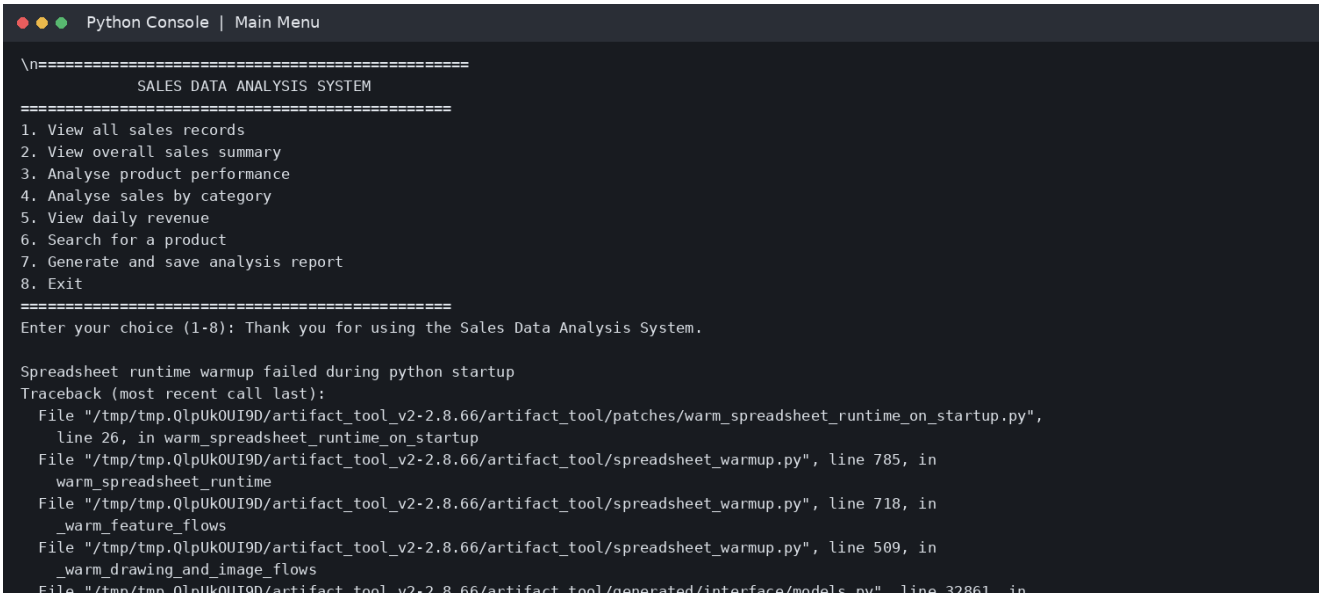


Figure 1. Main menu of the Sales Data Analysis System.

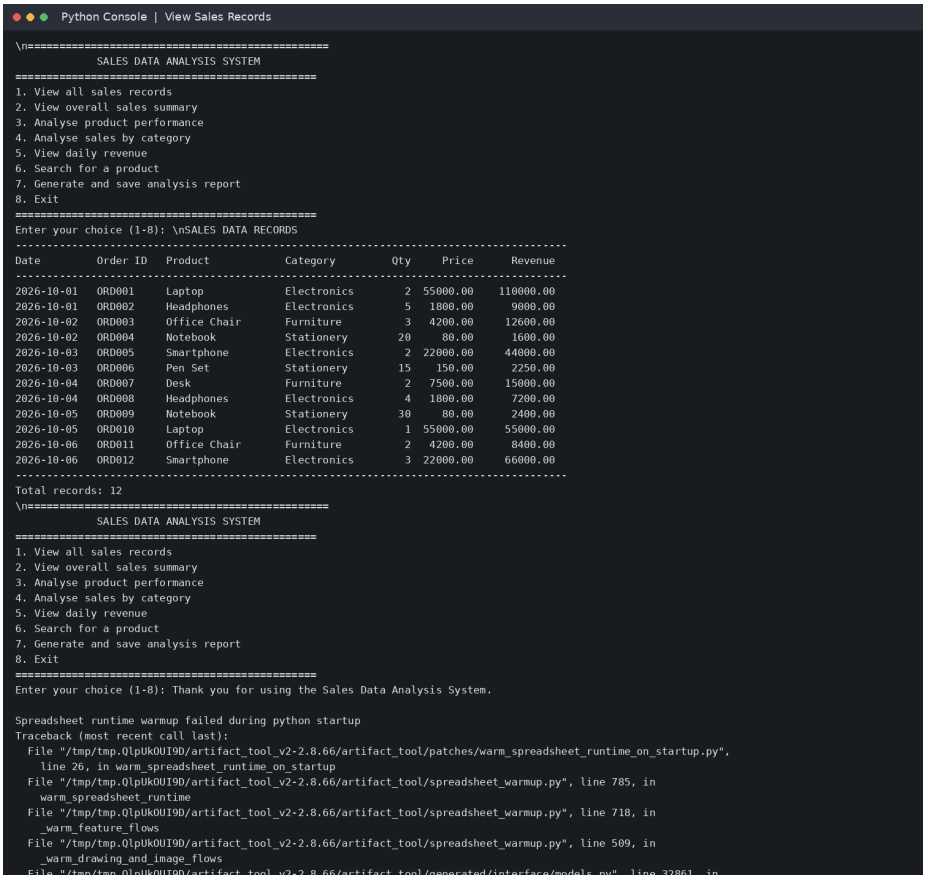


Figure 2. Display of all sales records.

Python Console | Overall Sales Summary

```
\n================================================
            SALES DATA ANALYSIS SYSTEM
================================================
1. View all sales records
2. View overall sales summary
3. Analyse product performance
4. Analyse sales by category
5. View daily revenue
6. Search for a product
7. Generate and save analysis report
8. Exit
================================================
Enter your choice (1-8): \nOVERALL SALES SUMMARY
Sales records       : 12
Unique orders       : 12
Units sold          : 89
Total revenue       : Rs. 333,450.00
Average order value : Rs. 27,787.50
\n================================================
            SALES DATA ANALYSIS SYSTEM
================================================
1. View all sales records
2. View overall sales summary
3. Analyse product performance
4. Analyse sales by category
5. View daily revenue
6. Search for a product
7. Generate and save analysis report
8. Exit
================================================
Enter your choice (1-8): Thank you for using the Sales Data Analysis System.

Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py",
    line 26, in warm_spreadsheet_runtime_on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in
    warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in
    _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in
    _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in
```

Figure 3. Overall sales summary including total revenue and units sold.

Python Console | Product Performance

```
\n================================================
            SALES DATA ANALYSIS SYSTEM
================================================
1. View all sales records
2. View overall sales summary
3. Analyse product performance
4. Analyse sales by category
5. View daily revenue
6. Search for a product
7. Generate and save analysis report
8. Exit
================================================
Enter your choice (1-8): \nPRODUCT PERFORMANCE
Product                Quantity Sold          Revenue
------------------------------------------------------
Laptop                             3    Rs. 165,000.00
Smartphone                         5    Rs. 110,000.00
Office Chair                       5     Rs. 21,000.00
Headphones                         9     Rs. 16,200.00
Desk                               2     Rs. 15,000.00
Notebook                          50      Rs. 4,000.00
Pen Set                           15      Rs. 2,250.00
Best-selling product (quantity): Notebook
Highest revenue product      : Laptop
\n================================================
            SALES DATA ANALYSIS SYSTEM
================================================
1. View all sales records
2. View overall sales summary
3. Analyse product performance
4. Analyse sales by category
5. View daily revenue
6. Search for a product
7. Generate and save analysis report
8. Exit
================================================
Enter your choice (1-8): Thank you for using the Sales Data Analysis System.

Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py",
    line 26, in warm_spreadsheet_runtime_on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in
    warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in
    _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in
    _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in
```

Figure 4. Product performance comparison and best-selling product.

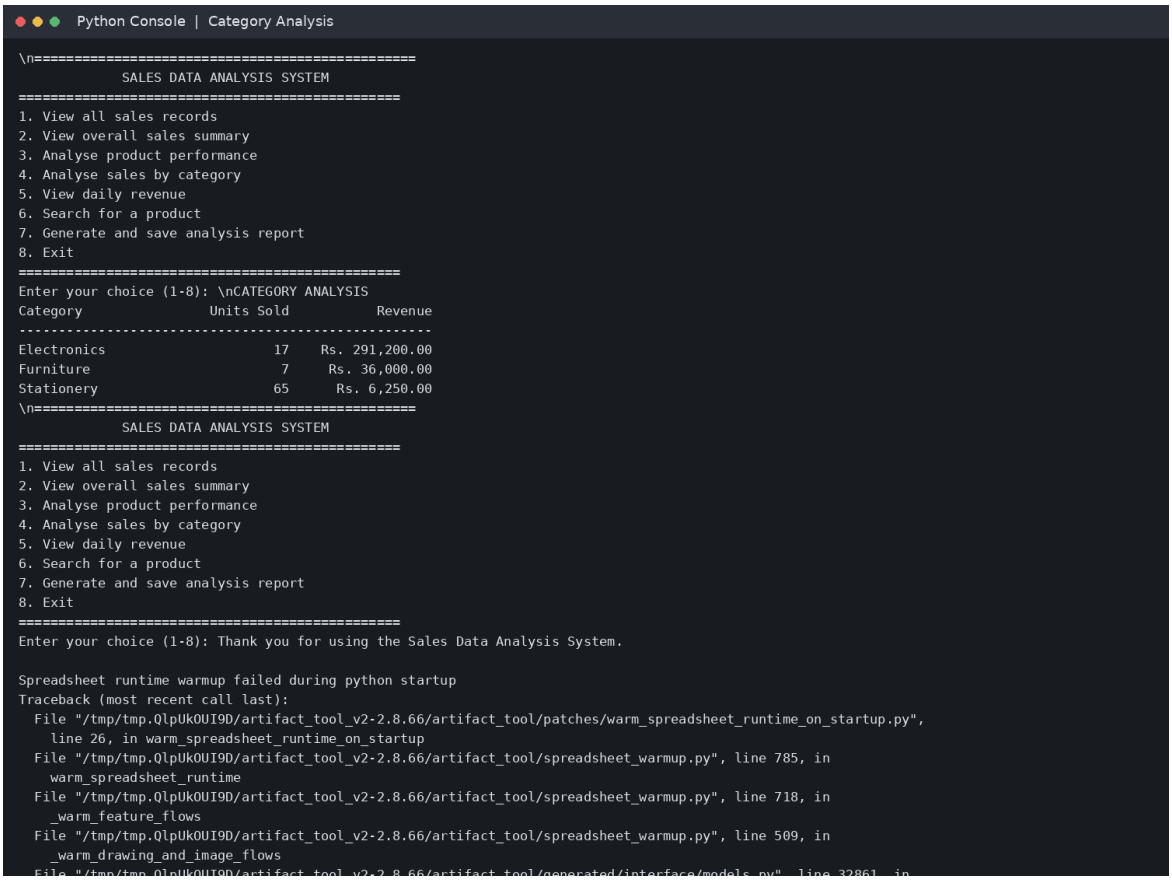


Figure 5. Category-wise quantity and revenue analysis.

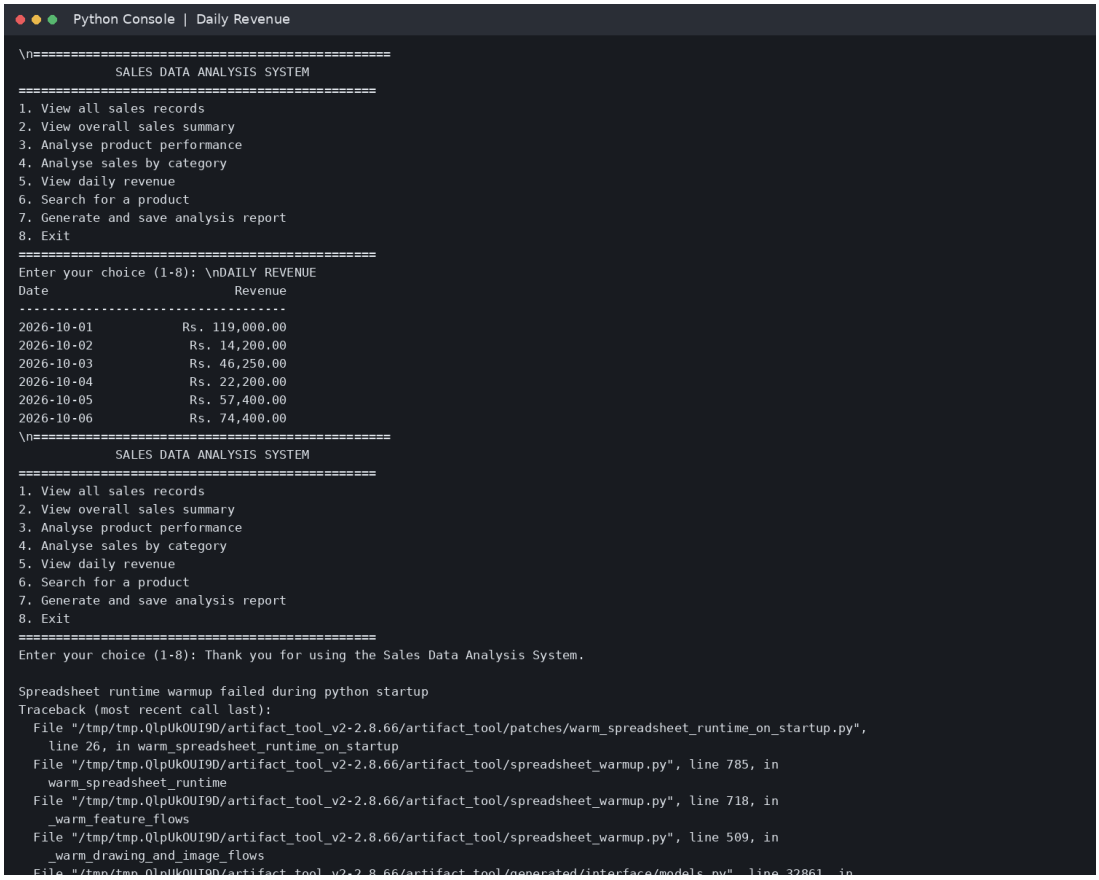


Figure 6. Daily revenue summary.

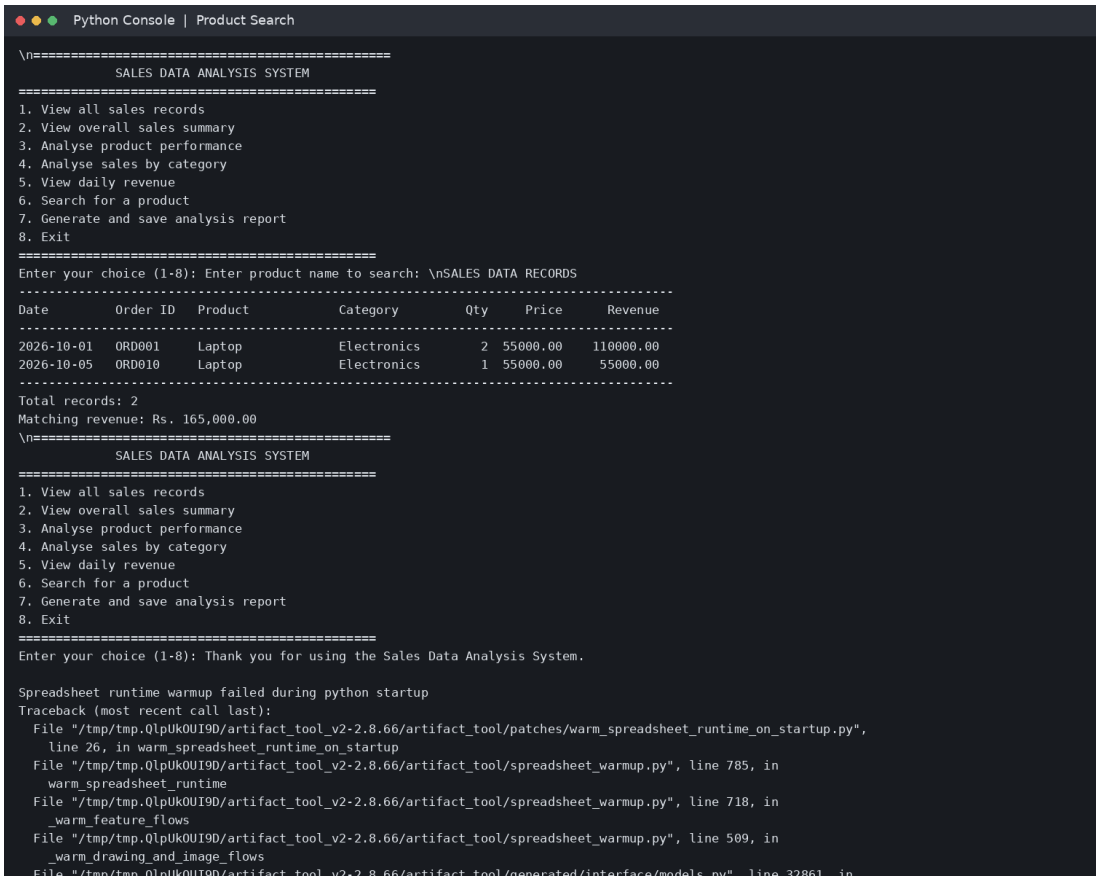


Figure 7. Product search example using Laptop.

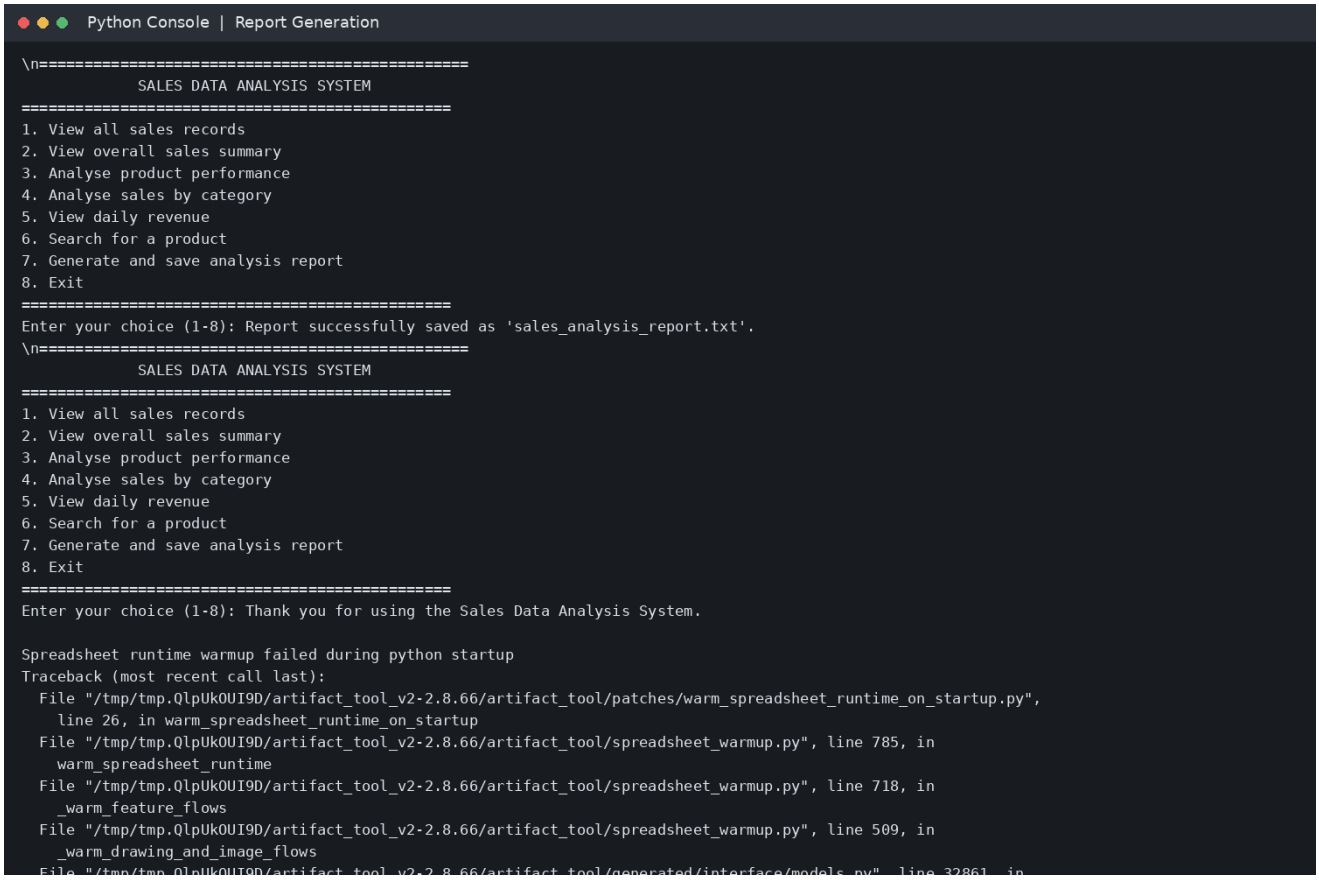


Figure 8. Successful generation of the analysis report.

11. TESTING AND RESULTS

Test ID	Test performed	Expected result	Result
T01	Launch program with valid CSV	Menu is displayed	Pass
T02	Select option 1	All valid sales rows are displayed	Pass
T03	Select option 2	Totals and average order value are calculated	Pass
T04	Select option 3	Product totals and leading products are shown	Pass
T05	Select option 4	Category totals are displayed	Pass
T06	Select option 5	Revenue is grouped by date	Pass
T07	Search for Laptop	Matching rows and revenue are displayed	Pass
T08	Select option 7	Text report is written to file	Pass
T09	Enter an unsupported menu number	Invalid-choice message is shown	Pass

The tests confirm that the core menu functions operate with the supplied sample dataset. The system calculates each record's revenue from quantity and unit price, then aggregates values according to the selected analysis. Invalid menu selections are handled with a message rather than terminating the program.

12. LIMITATIONS AND FUTURE SCOPE

The prototype uses a small local CSV dataset and a text-based interface. It does not provide user authentication, multi-user access, a database, graphical dashboards or automated data visualisation. The accuracy of results depends on the correctness and consistency of the source records.

Possible future improvements include a graphical user interface, charts using Matplotlib, database storage using SQLite or MySQL, date-range filters, export to Excel/PDF, additional validation rules, and larger datasets from an actual retail operation.

13. CONCLUSION

The Sales Data Analysis System demonstrates how Python can be used to convert structured sales records into meaningful business information. It automates revenue calculations, product and category comparisons, daily summaries and report generation. The project provides practical experience with functions, collections, loops, conditional logic, file handling, exception handling and standard-library modules. It meets its learning objective of applying Python programming concepts to a realistic data-processing task.

14. REFERENCES

1. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
2. Python Software Foundation, csv — CSV File Reading and Writing: https://docs.python.org/3/library/csv.html
3. Python Software Foundation, datetime — Basic Date and Time Types: https://docs.python.org/3/library/datetime.html

4. Python Software Foundation, collections — Container Datatypes: https://docs.python.org/3/library/collections.html